

Advanced Natural Language Processing

A History of Deep Learning

Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes – Course year 2026-2027



Maastricht University

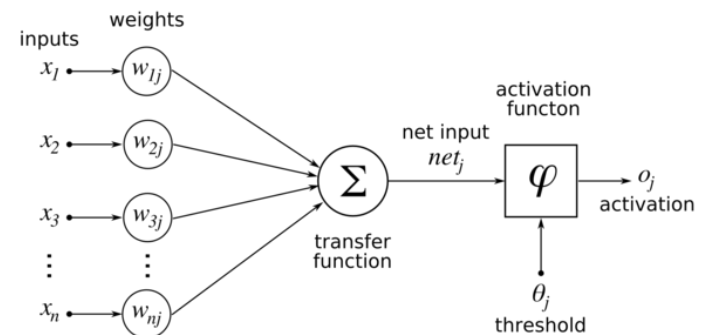
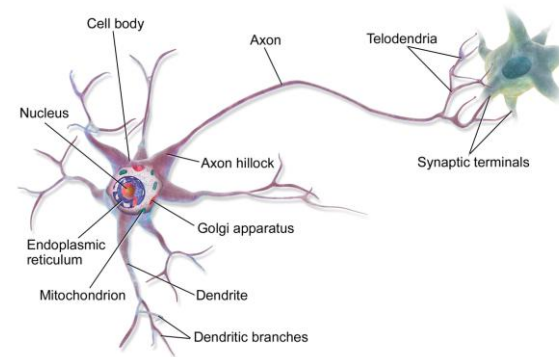
1943: First interest Artificial Neurons *

An artificial neuron is a mathematical function conceived as a model of biological neurons, a neural network.

Artificial neurons are elementary units in an artificial neural network.

* Also referred to as perceptron (two layers), Artificial Neural Network (3 layers with back-propagation) and deep learning (>3 layers with convolution) and Long Short-Term Memory Models (LSTM), bidirectional-LSTM or Transformers (BERT) for recurrent neural models.

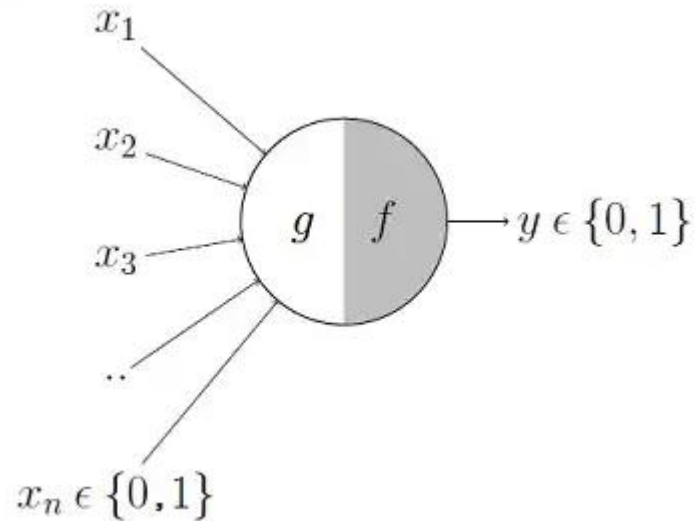
Source: Wikipedia



1943: McCulloch-Pitts Neuron

- The first computational model of a neuron was proposed by Warren McCulloch (neuroscientist) and Walter Pitts (logician) in 1943.
- Inputs and Outputs are all Boolean values $\{0,1\}$. No other input.
- Can implement AND, OR, NOT, but not XOR.
- No weights per input.
- No trigger function, just a sum of the inputs.

Teaching note: No weights + no trigger function = every input counts equally and the threshold is fixed by hand — the Perceptron (slide 8) adds exactly these two missing pieces.

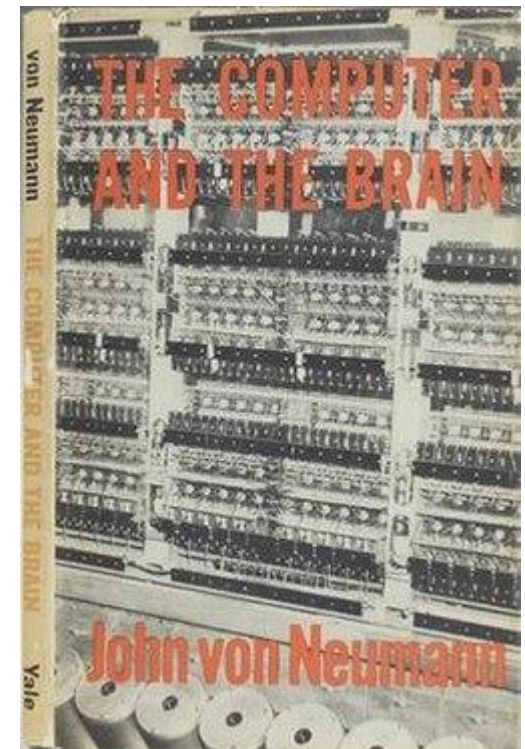


$$g(x_1, x_2, x_3, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n x_i$$

$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{if } g(\mathbf{x}) \geq \theta \\ 0 & \text{if } g(\mathbf{x}) < \theta \end{cases}$$

Other Brain-like inspirations in Computer Science

- *The Computer and the Brain* is an unfinished book by mathematician John von Neumann, begun shortly before his death and first published in 1958.
- “How the brain can be viewed as a computing machine.”
- The book is speculative in nature, but von Neumann discusses several important differences between brains and computers of his day (such as processing speed and parallelism), as well as suggesting directions for future research.



Alan Turing

Most people associate Turing with symbolic AI (Turing machine, logic), but in the late 1940s and early 1950s he explored a second line of thought that today we would call neural-network learning.

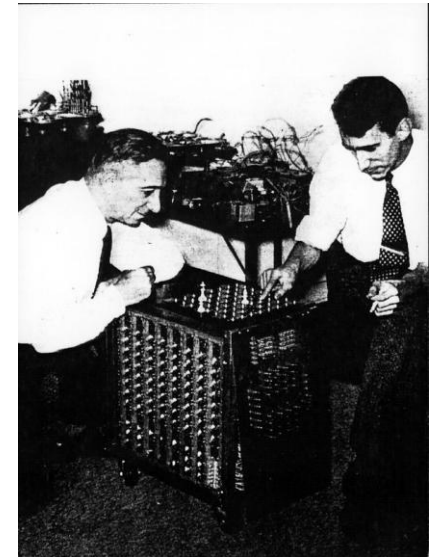


Concept	Deep learning version	Turing's version
Neural network with binary units	McCulloch–Pitts neuron / perceptron	"A-type unorganized machine"
Trainable connections	Learning weights	"B-type unorganized machine"
Supervised learning	Loss + gradient update	Reward/punish modification of connections
Optimization / search	Evolutionary search / NAS	Mutation + selective improvement
Random initialization	Xavier/He initialization	Start random, then train

"Intelligent Machinery" (National Physical Laboratory, 1948 — not published until 1968)

Claude Shannon's Chess computers & Maze Robots

- Shannon known for information theory (1948), which gave us entropy, bits, and the mathematical idea of information.
- Shannon also explored learning machines, neural nets, and robotics — long before AI became a discipline.
- Shannon loved randomness. He believed that randomness + selection can produce intelligence.



Concept (Shannon)	Connection to modern deep learning
Entropy	loss functions, uncertainty estimation
Information channel capacity	limits of model compression and generalization
Encoding/decoding theory	autoencoders, representation learning

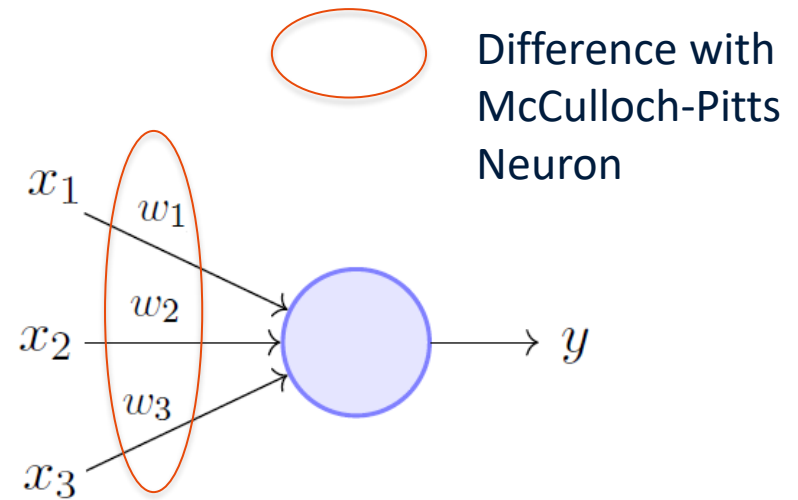
Key Pioneers of Deep Learning 40-50s

Person	Rough period	Contribution / relevance to deep learning ancestors
Warren McCulloch & Walter Pitts (1943)	1943	Formalized the first mathematical model of an artificial neuron (“A Logical Calculus of Ideas Immanent in Nervous Activity”). Introduced thresholded neurons and networks as logic machines. Foundational to ANNs.
John von Neumann	late 1940s–1950s	Explored neural-style architectures, redundancy, and reliability of computing . Book: <i>The Computer and the Brain</i> (posthumous, 1958).
Alan Turing	1940s–1950s	Proposed learning machines , even speculated about reinforcement-type training and evolution-based learning (<i>Intelligent Machinery</i> , 1948).
Claude Shannon	late 1940s–1950s	Founder of information theory. Also worked on neural nets and maze-solving robots (“Theseus”). Discussed computation and learning.
Donald Hebb (1949)	1949	Introduced Hebbian learning (“neurons that fire together wire together”). This is the conceptual ancestor of weight adaptation rules .
Norbert Wiener	1948	Founder of cybernetics . Focus on feedback loops and control systems—central to early views of learning and adaptation.
Frank Rosenblatt	late 1950s	Invented the perceptron (1957/58) → <i>first trainable neural network</i> . Hardware and simulations, defense-funded.
Oliver Selfridge	1959	Proposed the Pandemonium architecture (early multi-level pattern recognition)—a precursor to layered feature extraction.

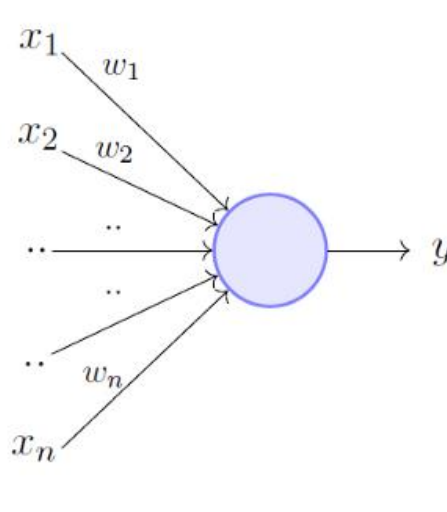
1958: The Perceptron

Basic units of neural networks:

- Input:
 - Feature vector of real input values.
 - Weights
- Activation Function:
 - Sigmoid, tanh, ..
- Output:
 - Boolean



Perceptron Model (Minsky-Papert in 1969)



$$y = 1 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i \geq \theta$$

$$= 0 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i < \theta$$

Rewriting the above,

$$y = 1 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i - \theta \geq 0$$

$$= 0 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i - \theta < 0$$

1958: Perceptron: the Expectations were High

Navy officers demonstrating a preliminary form of the device in Washington said they hesitated to call it a machine because it is so much like a "human being without life."

human brain. Like humans, Perceptron will make mistakes at first, "but it will grow wiser as it gains experience," he said.

In principle, Dr. Rosenblatt said, it would be possible to build Perceptrons that could reproduce themselves on an assembly line and which would be "conscious" of their existence.

The Navy said it would use the principle to build the first Perceptron "thinking machines" that will be able to read or write.

"Conscious, Thinking, Grow Wiser, Like a Human Being, big words for such a simple device

The New York Times

SUNDAY, JULY 13, 1958

ways.

There are "inadequate and intriguing hints," he said, "that the synthesis and use of large molecules of nucleic acids (chemicals that con-

pressed the hope that it will prove useful in the treatment of bladder cancer for the general population, when the chemical becomes generally available.

Electronic 'Brain' Teaches Itself

The Navy last week demonstrated the embryo of an electronic computer named the Perceptron which, when completed in about a year, is expected to be the first non-living mechanism able to "perceive, recognize and identify its surroundings without human training or control." Navy officers demonstrating a preliminary form of the device in Washington said they hesitated to call it a machine because it is so much like a "human being without life."

Dr. Frank Rosenblatt, research psychologist at the Cornell Aeronautical Laboratory, Inc., Buffalo, N. Y., designer of the Perceptron, conducted the demonstration. The machine, he said, would be the first electronic device to think as the human brain. Like humans, Perceptron will make mistakes at first, "but it will grow wiser as it gains experience," he said.

The first Perceptron, to cost about \$100,000, will have about 1,000 electronic "association cells" receiving electrical impulses from an eyelike scanning device with 400 photocells. The human brain has ten billion responsive cells, including 100,000,000 connections with the eye.

Difference Recognized

The concept of the Perceptron was demonstrated on the Weather Bureau's \$2,000,000 IBM 704 computer. In one experiment, the 704 computer was shown 100 squares situated at random either on the left or the right side of a field. In 100 trials, it was able to "say" correctly ninety-seven times whether a square was situated on the right or left. Dr. Rosenblatt said that after having seen only thirty to forty squares the device had learned to

recognize the difference between right and left, almost the way a child learns.

When fully developed, the Perceptron will be designed to remember images and information it has perceived itself, whereas ordinary computers remember only what is fed into them on punch cards or magnetic tape.

Later Perceptrons, Dr. Rosenblatt said, will be able to recognize people and call out their names. Printed pages, longhand letters and even speech commands are within its reach. Only one more step of development, a difficult step, he said, is needed for the device to hear speech in one language and instantly translate it to speech or writing in another language.

Self-Reproduction

In principle, Dr. Rosenblatt said, it would be possible to build Perceptrons that could reproduce themselves on an assembly line and which would be "conscious" of their existence.

Perceptron, it was pointed out, needs no "priming." It is not necessary to introduce it to surroundings and circumstances, record the data involved and then store them for future comparison as is the case with present "mechanical brains." It literally teaches itself to recognize objects the first time it encounters them. It uses a camera-eye lens to scan objects or survey situations, and an electrical impulse system, patterned point-by-point after the human brain does the interpreting.

The Navy said it would use the principle to build the first Perceptron "thinking machines" that will be able to read or write.

1958: Perceptron

Teaching note: This exact XOR failure is what Minsky & Papert formalized mathematically one slide later — triggering the first AI winter.

- Drawback:
 - Only linear separable functions
 - E.g:
 - No XOR: there is no line you can draw to separate the **blue** from the **red** data points

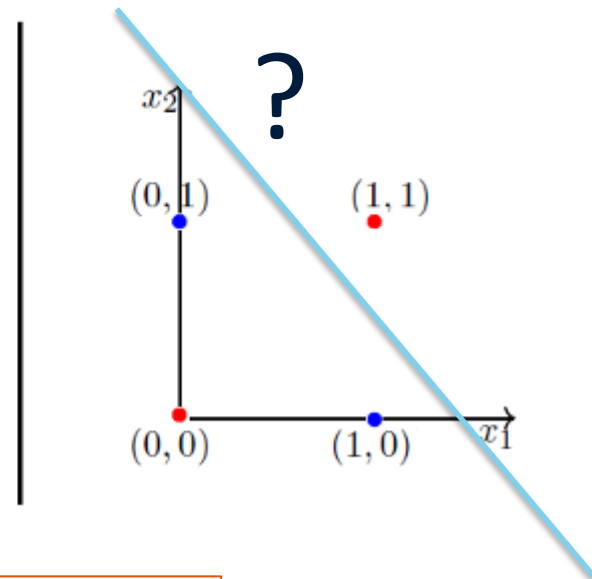
x_1	x_2	XOR	
0	0	0	$w_0 + \sum_{i=1}^2 w_i x_i < 0$
1	0	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
0	1	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
1	1	0	$w_0 + \sum_{i=1}^2 w_i x_i < 0$

$$w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \implies w_0 < 0$$

$$w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \implies w_2 > -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \implies w_1 > -w_0$$

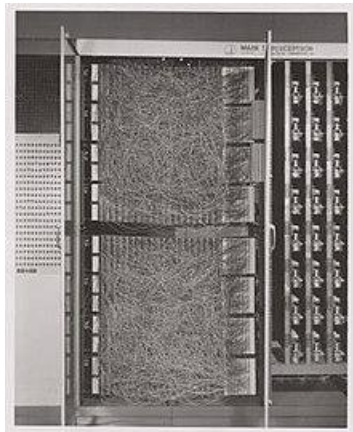
$$w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 \implies w_1 + w_2 < -w_0$$



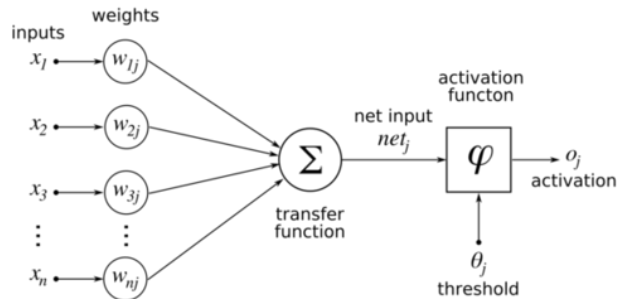
1969: Perceptron's could only solve linear problems: start of a long AI (ANN) winter ...



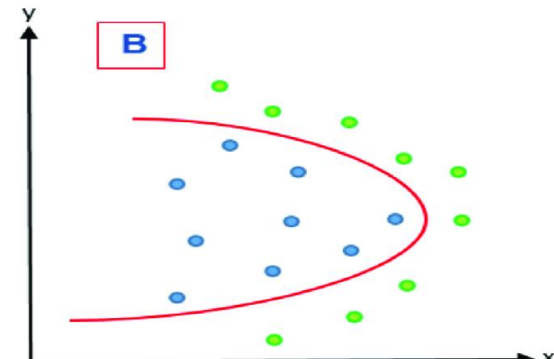
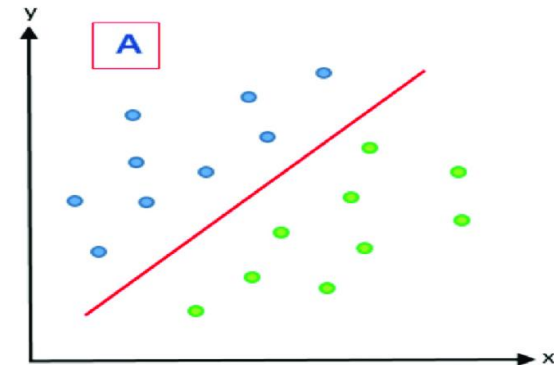
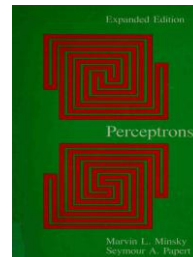
1958-1969
Frank Rosenblatt
invented the
perceptron. Human
brain like computing.



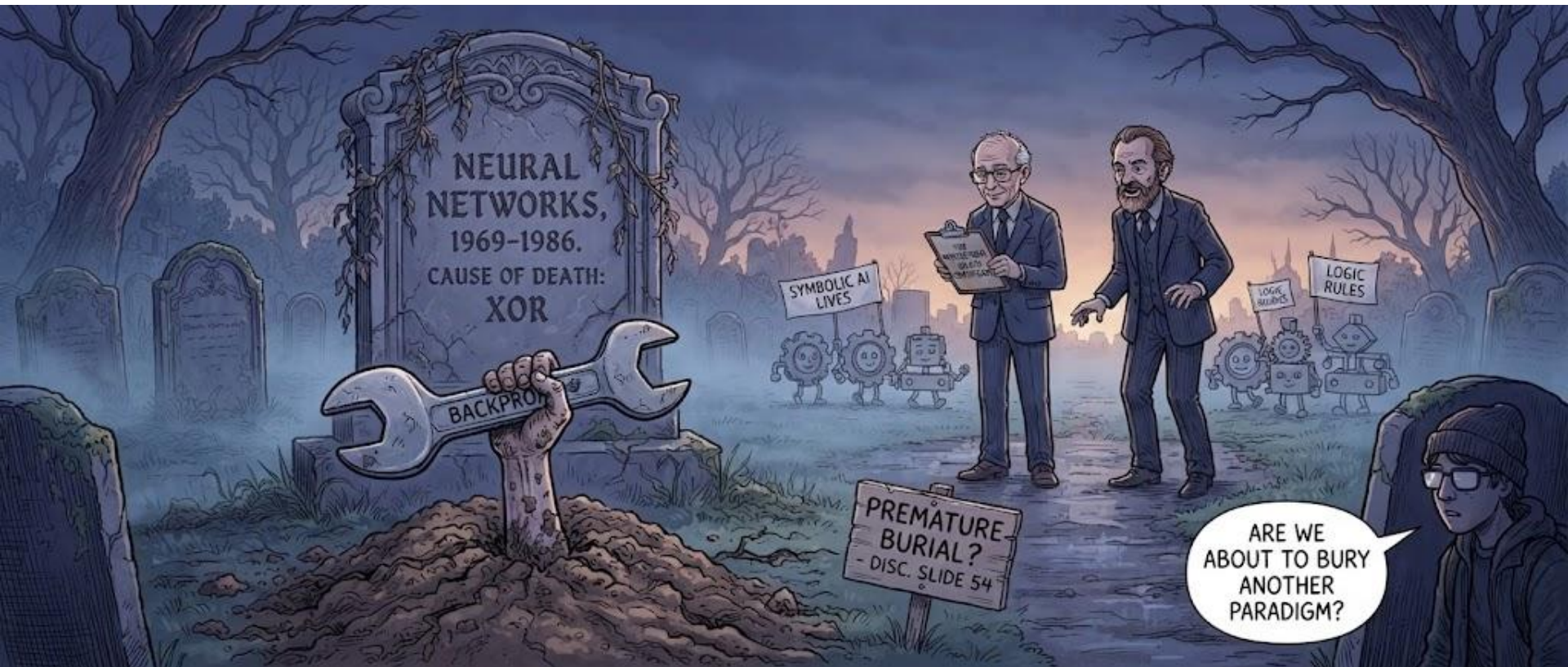
The "perceptron" hardware implementation



1969: Marvin Minsky & Simon
Papert ended the first AI
neural summer by showing
the perceptron was a simple
"linear classifier" that could
not solve non- linear"
problems.



So, this led to the burial of the Perceptron and the first Neural Network winter.



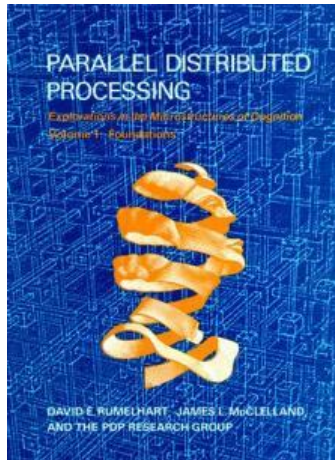
We had to wait for Back Propagation



Maastricht University

Teaching note: The missing piece was training multi-layer networks — backpropagation (next slides) is precisely what Minsky & Papert's critique showed was needed but unavailable in 1969.

1988: Back-propagation: another AI (neural) summer



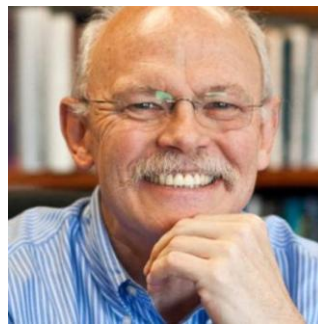
- 3-layer networks
- Could learn from data
- Also capable of capturing non-linear relations

Progress in optical character recognition and other simple pattern recognition tasks.

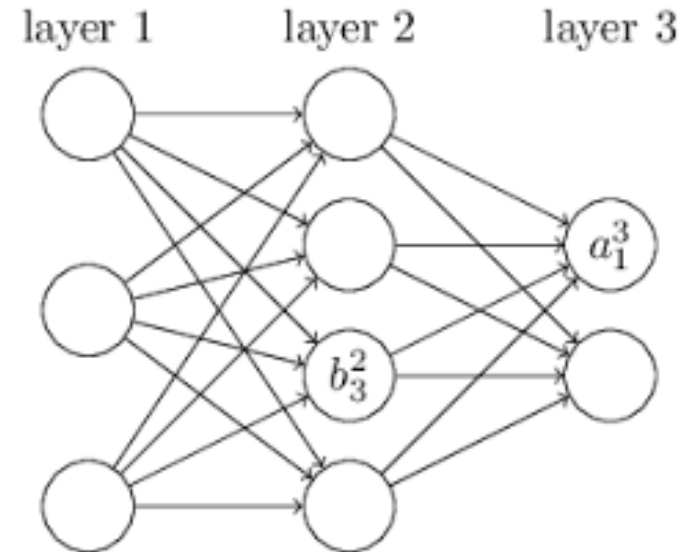
The sky seemed to be the limit

...

D. Rumelhart



J. McClelland



See also:

- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536. DOI: 10.1038/323533a0.
- Parallel Distributed Processing. Volume 1: Foundations, D. Rumelhart, J. McClelland, and PDP Research Group (Eds.). MIT Press, Cambridge, MA, (1986)

Why is Back-propagation Still Relevant?

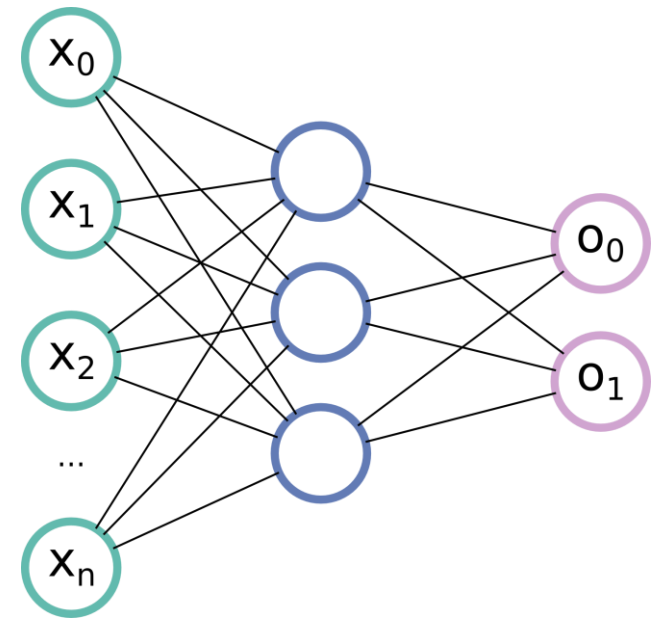
- The algorithm we now simply call backpropagation was popularized by Rumelhart, Hinton & Williams in the 1986–1988 Parallel Distributed Processing (PDP) books.
- It was originally presented for training multi-layer perceptrons (MLPs) with differentiable activation functions.
- Backprop provided an efficient way to compute gradients through a network with many layers.

Even though today's networks look very different (CNNs, RNNs, Transformers), their training loop is still based on the same principle!



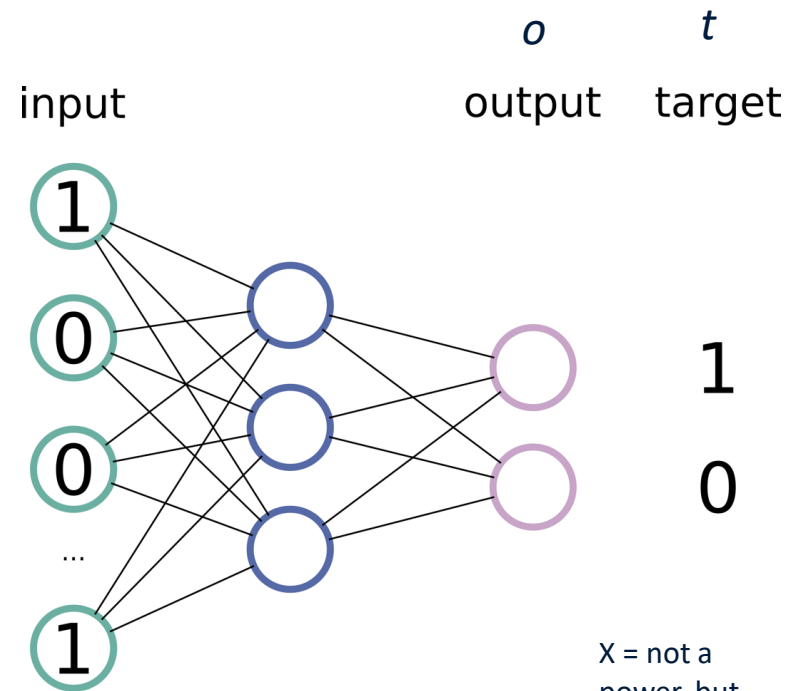
BP: A Closer look

- Connect several layers of perceptrons (aka neurons)
- Nodes:
 - Input nodes
 - Hidden nodes
 - Output nodes
- Edges:
 - Weighted connections between nodes
- Parameters:
 - Weights of edges



Error Function

- Supervised learning:
 - Compare output o to desired output t over n neurons and x data points.
- Measure
 - Compare difference between output and target:
 - Mean square error (used for regression = continuous quantity))
 - Cross entropy (used for classification = discrete classes)



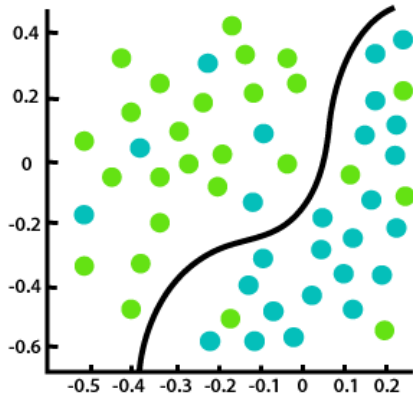
$$E = \frac{1}{2} \sum_x \sum_{i=1}^n (t_i^x - o_i^x)^2$$

X = not a power, but the x-the data point.

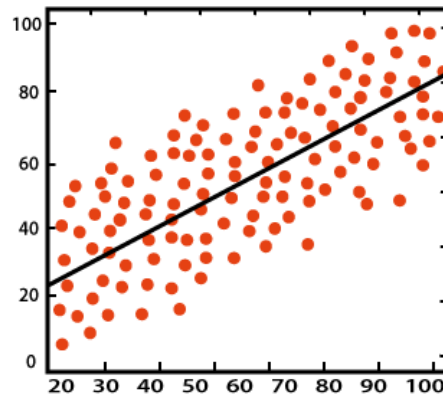
$$E = \sum_x \sum_{i=1}^n t_i^x * \ln(o_i^x)$$

Value for neuron i

Regression versus Classification



Classification



Regression

$$E = \sum_x \sum_{i=1}^n t_i^x * \ln(o_i^x) \quad E = \frac{1}{2} \sum_x \sum_{i=1}^n (t_i^x - o_i^x)^2$$

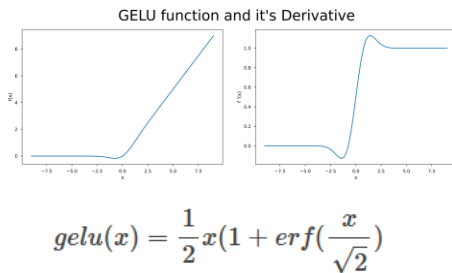
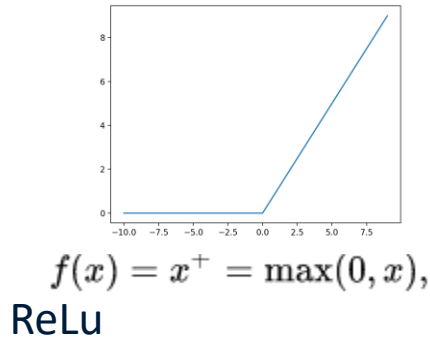
- The (mean) cross entropy is a measure of difference between two *discrete* probability distributions. In particular the categorical cross entropy is used as the loss function where the target variable is binary (0, 1) with some probability (0..1).

- The mean squared error is a measure of distance (expressed as the variance) between two sets of real (*continuous*) values (-inf .. +inf).

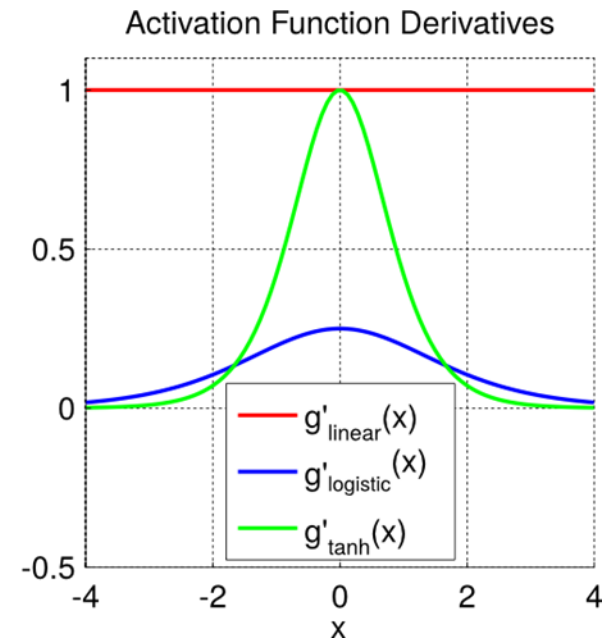
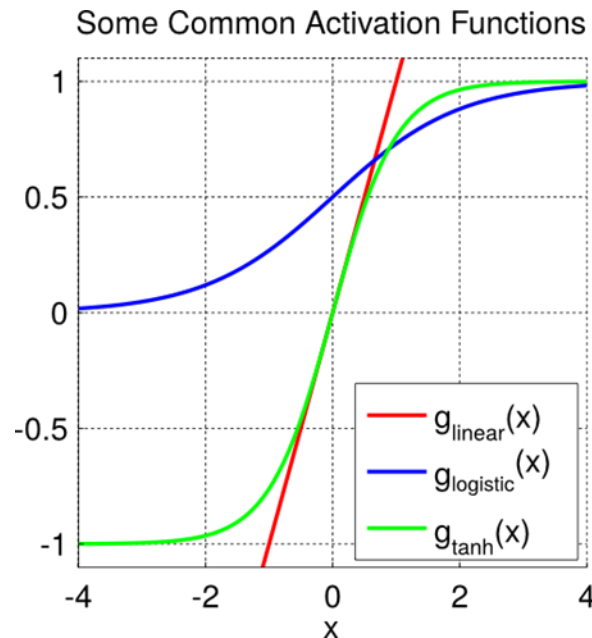
Activation Functions

Teaching note: Sigmoid/tanh saturate (near-zero gradient at the extremes); ReLU/GELU don't — this single property is a major reason deep networks became trainable at all (see slide 51).

- Hidden layers traditionally used a sigmoid function, but today is more varied, with rectifier (ReLU-rectified linear unit) being common.



GELU



$$g_{\text{linear}}(z) = z$$

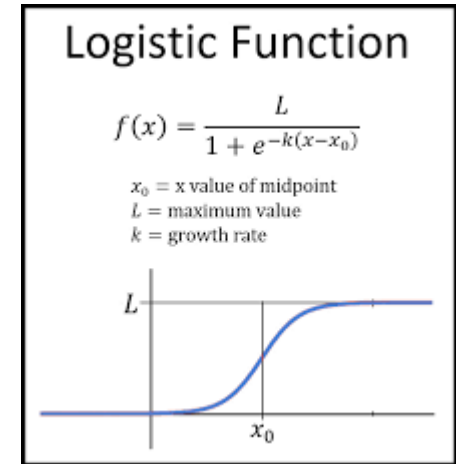
$$g_{\text{logistic}}(z) = \frac{1}{1+e^{-z}}$$

$$g_{\text{tanh}}(z) = \frac{\sinh(z)}{\cosh(z)} = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

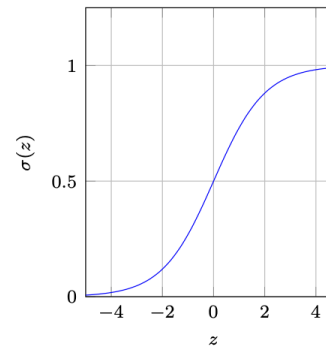
Output Layer

$$S(x) = \frac{1}{1 + e^{-x}}$$

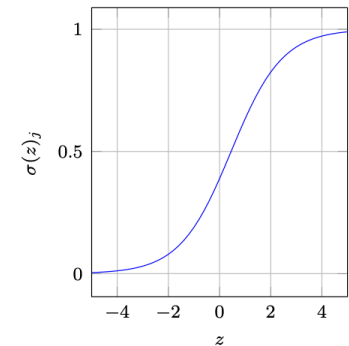
- For classification the last layer is usually the logistic aka sigmoid function for binary classification, and
- softmax (softargmax) for multi-class classification,



$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$



(a) Sigmoid activation function.



(b) Softmax activation function.

Softmax

- Used for multi-class classification.
- The softmax function is a function that turns a vector of K real values into a vector of K real values that sum to 1.
- The input values can be positive, negative, zero, or greater than one, but the softmax transforms them into values between 0 and 1,
- Softmax outcome can be interpreted as probabilities.
- Can also be seen as normalization.

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

$\vec{z}$

The input vector to the softmax function, made up of $z = (z_0, \dots, z_K)$

z_i

All the z_i values are the elements of the input vector to the softmax function, and they can take any real value, positive, zero or negative.

e^{z_i}

The standard exponential function is applied to each element of the input vector. This gives a positive value above 0, which will be very small if the input was negative, and very large if the input was large. However, it is still not fixed in the range (0, 1) which is what is required of a probability.

$\sum_{j=1}^K e^{z_j}$

ensures that all the output values of the function will sum to 1 and each be in the range (0, 1), thus constituting a valid probability distribution.

K

The number of classes in the multi-class classifier.

Softmax versus Sigmoid

The softmax function and the sigmoid function are similar. The softmax operates on a vector while the sigmoid takes a scalar.

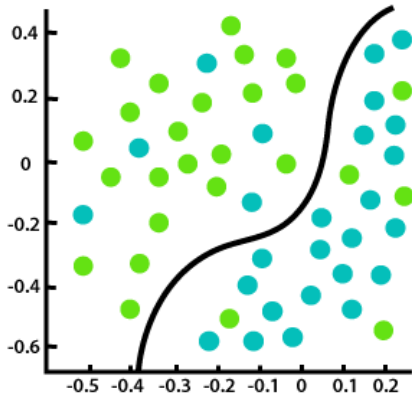
- Single classification (scalar): sigmoid.
- Multi-class (vector): softmax.

$$\text{Softmax } \sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

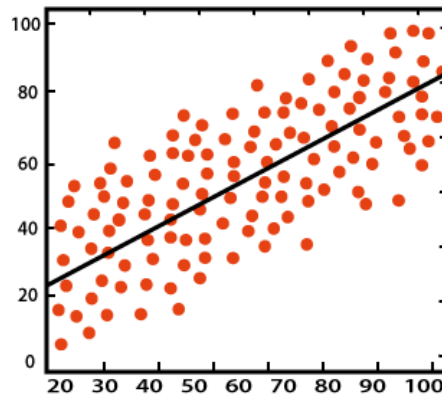
$$\text{Sigmoid } S(x) = \frac{1}{1 + e^{-x}}$$

for two classes, softmax = sigmoid (you can try this yourself).

The whole picture ...



Classification



Regression

$$E = \sum_x \sum_{i=1}^n t_i^x * \ln(o_i^x)$$



$$\text{Softmax } \sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

$$\text{Sigmoid } S(x) = \frac{1}{1 + e^{-x}}$$

$$E = \frac{1}{2} \sum_x \sum_{i=1}^n (t_i^x - o_i^x)^2$$



Single classification: sigmoid.
Multi-class: softmax.

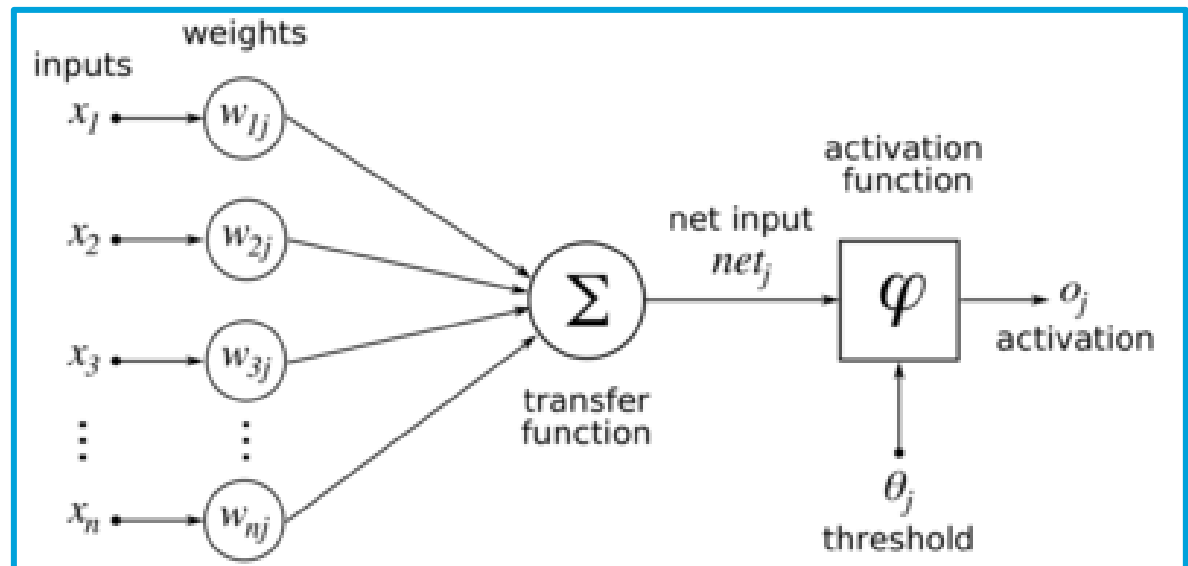
- The (mean) cross entropy is a measure of difference between two *discrete* probability distributions. In particular the categorical cross entropy is used as the loss function where the target variable is binary (0, 1) with some probability (0..1).

- The mean squared error is a measure of distance (expressed as the variance) between two sets of real (*continuous*) values (-inf .. +inf).

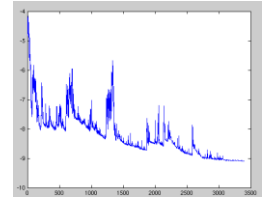
Back Propagation

Teaching note: “Differentiable everywhere” is the hard constraint that ruled out McCulloch-Pitts' step function and shaped every activation function choice since (slide 20).

- Goal:
 - Minimize error
 - So: find weights w_1, w that minimize prediction error.
- All functions are non-linear and need to be differentiable (this is why activation functions need to be continuous and differentiable).
- Convergence requires derivative (changes) of error with respect to weight changes needs to go to zero.



Gradient Descent (on a linear model)



Goal:

We want to find weights w_1, w_2 that minimize prediction error.

Suppose we want to fit a straight line:

$$\hat{y} = w_1 + w_2 x$$

For a training example (x_i, y_i) , the prediction is:

$$\hat{y}_i = w_1 + w_2 x_i$$



Define error (loss function)

Using **Mean Squared Error** (for SGD: we look at one example at a time):

$$E = (\hat{y}_i - y_i)^2$$

Substitute prediction:

$$E = (w_1 + w_2 x_i - y_i)^2$$

Gradient descent philosophy:

Move weights in direction of **steepest decrease** of error.

Update rule:

$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

where

- η is the learning rate,
- $\frac{\partial E}{\partial w_j}$ is how much the weight influences error.

Compute partial derivatives

For w_1 :

$$\frac{\partial E}{\partial w_1} = \frac{\partial}{\partial w_1} (w_1 + w_2 x_i - y_i)^2 = 2(w_1 + w_2 x_i - y_i)$$

For w_2 :

$$\frac{\partial E}{\partial w_2} = 2(w_1 + w_2 x_i - y_i) x_i$$

Thus the update step becomes:

$$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} - \eta \begin{bmatrix} 2(w_1 + w_2 x_i - y_i) \\ 2x_i(w_1 + w_2 x_i - y_i) \end{bmatrix}$$

Neural Network: Forward Pass

A neuron computes a **weighted sum** ("net input"):

$$\text{net}_j = \sum_{k=1}^n w_{kj} o_k$$

Where:

- o_k is the output of neuron k (or input feature),
- w_{kj} is the weight from k to j .

Then we apply an activation function:

$$o_j = \varphi(\text{net}_j)$$

For example, sigmoid activation:

$$\varphi(z) = \frac{1}{1 + e^{-z}}$$

Goal:

Find how the error changes when a weight changes.

We want:

$$\frac{\partial E}{\partial w_{ij}}$$

Using the chain rule:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_j} \cdot \frac{\partial o_j}{\partial \text{net}_j} \cdot \frac{\partial \text{net}_j}{\partial w_{ij}}$$

Interpretation:

Term	Meaning
$\frac{\partial E}{\partial o_j}$	how changing neuron output affects error
$\frac{\partial o_j}{\partial \text{net}_j}$	slope of activation function
$\frac{\partial \text{net}_j}{\partial w_{ij}}$	how weight affects input to neuron

Compute the easy one first:

$$\frac{\partial \text{net}_j}{\partial w_{ij}} = o_i$$

Because:

$$\text{net}_j = \sum_k w_{kj} o_k$$

Only w_{ij} contributes to the derivative → its coefficient is o_i .

Back Propagation (the chain rule)

Delta (δ_j) and the final update rule

We define:

$$\delta_j = \frac{\partial E}{\partial o_j} \cdot \frac{\partial o_j}{\partial \text{net}_j}$$

So the weight gradient becomes simply:

$$\frac{\partial E}{\partial w_{ij}} = o_i \delta_j$$

This is **backpropagation**.

*With $n > 0$ Always
decreases $\rightarrow$ convergence*

$$\Delta w_{ij} = -\eta o_i \delta_j$$

If j is an output neuron, and we use squared error:

$$E = \frac{1}{2} (o_j - t_j)^2$$

Then:

$$\delta_j = (o_j - t_j) o_j (1 - o_j)$$

Forward pass computes outputs.
Backward pass computes blame (gradient).

Each weight is updated based on:

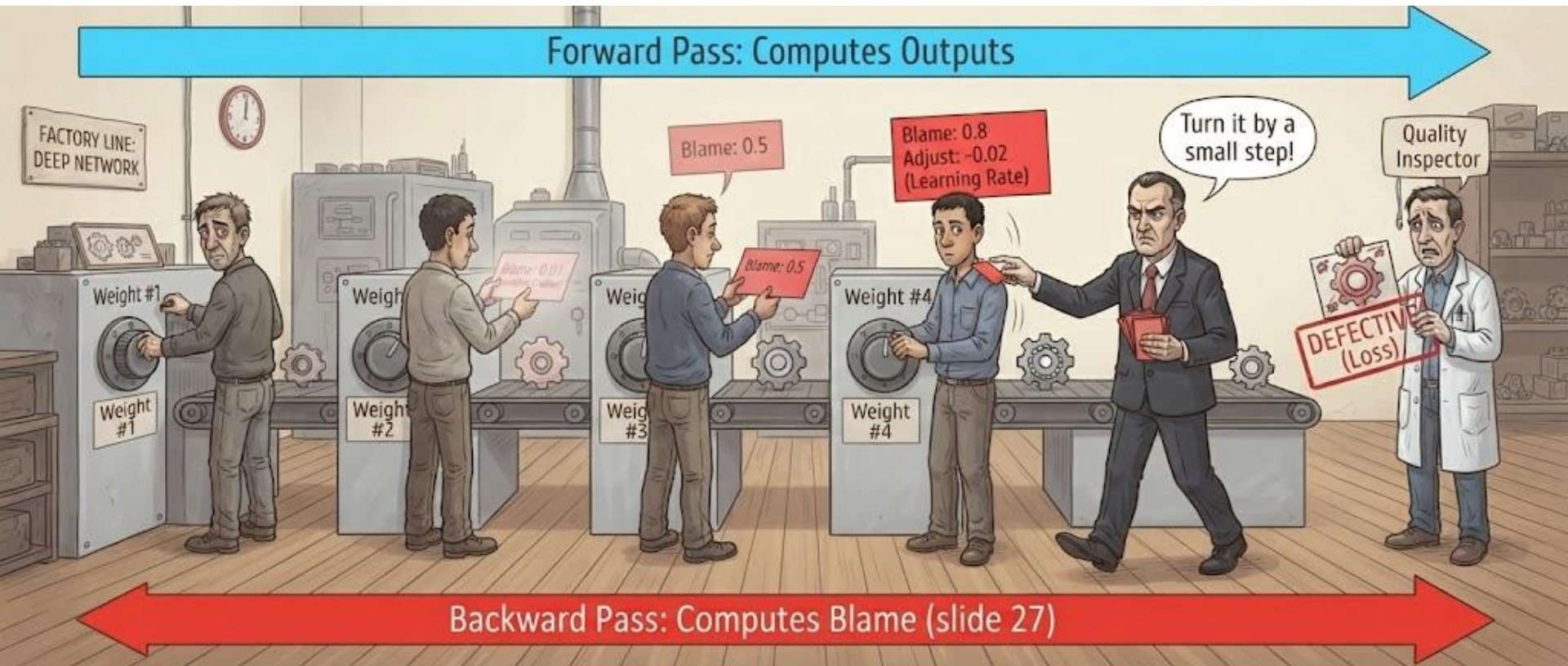
1. the input activating the weight o_i
2. the responsibility of that neuron δ_j
3. multiplied by learning rate η

Hidden Neurons:

$$\delta_j = \left(\sum_{k \in \text{next layer}} w_{jk} \delta_k \right) o_j (1 - o_j)$$

(The error is weighted sum of downstream errors.)

Another look at Back Propagation

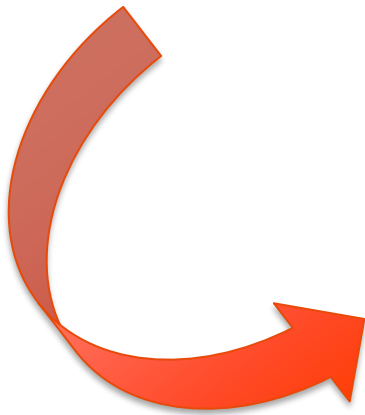


"Blame" is a plain-language word for the gradient: the backward pass computes blame, and each hidden neuron has a responsibility value δ (delta).

Variations on Back Propagation

There are many variations on backpropagation and gradient descent. They differ in:

- How the gradients are computed (e.g., non-linear activation, second-order methods)
- How weights are updated (optimizers)
- What loss function is optimized (cross-entropy, hinge loss, etc.)
- How data is processed (batch vs. mini-batch vs. stochastic)



Backprop = chain rule + gradient descent.

Variations come from changing:

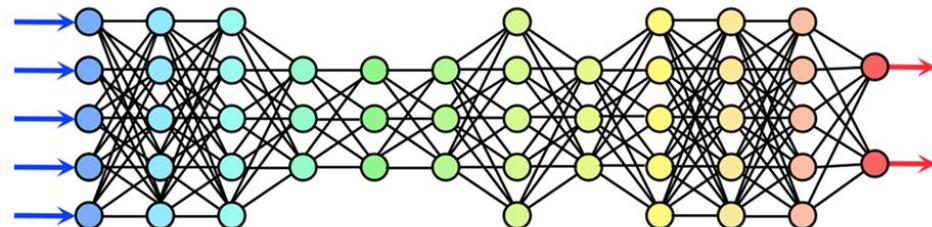
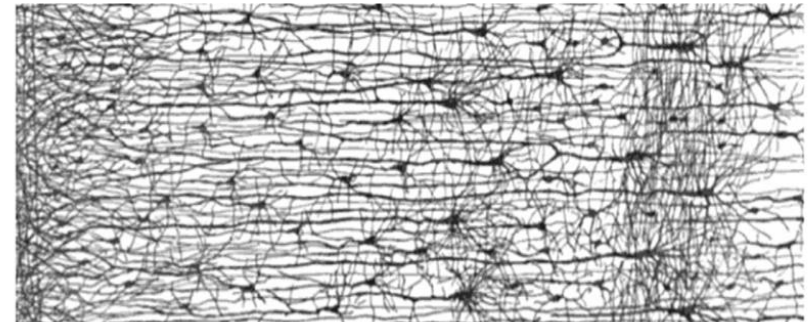
- (1) how gradients are computed → backprop variants
- (2) how gradients are applied → optimization algorithms
- (3) what we minimize → loss functions

Promising, but ...

Teaching note: “We knew we needed more layers but couldn't train them” — this exact bottleneck is what Hinton's 2012 breakthrough (slide 36) finally cracked.

- Back-propagation did not scale.
- It was often slow and unreliable (same input, different output).
- Success was very dependent on training data. Not enough “generalization”.

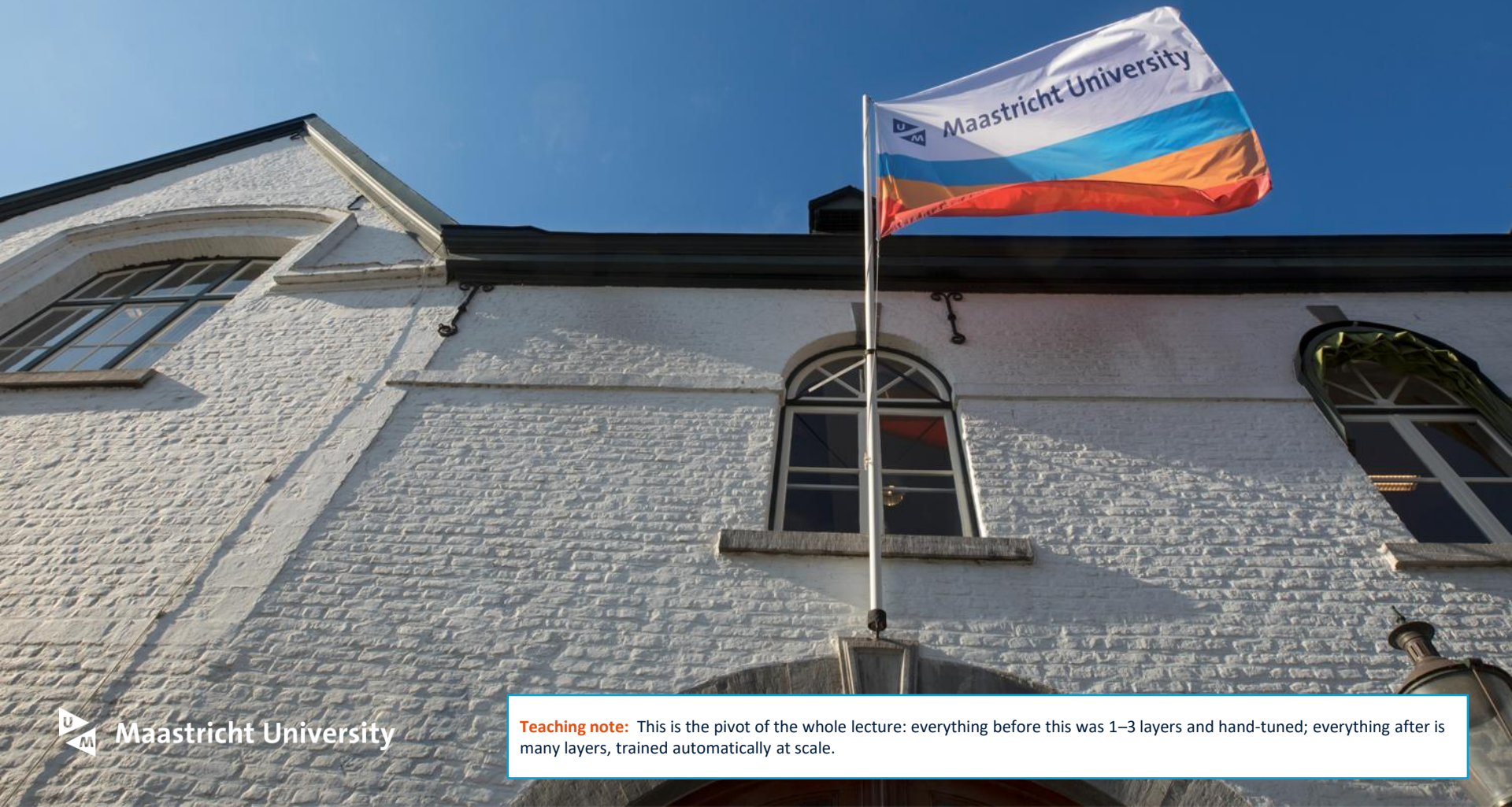
Intuitively, we all knew that we needed more hidden layers to reach better abstraction and generalization. Just like in the human brain. However, we were not able to train models with more than 1 hidden layer.



Break

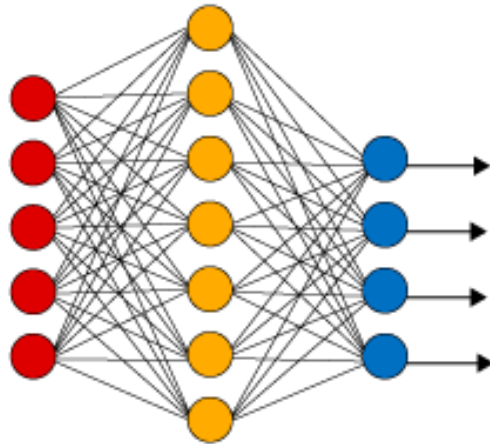


Deep Learning



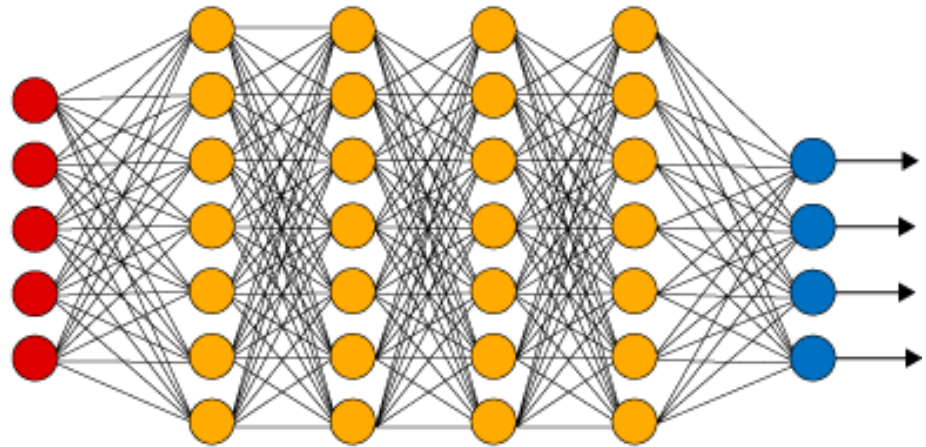
So, how did we go from Back Propagation to Deep Learning?

Simple Neural Network



● Input Layer

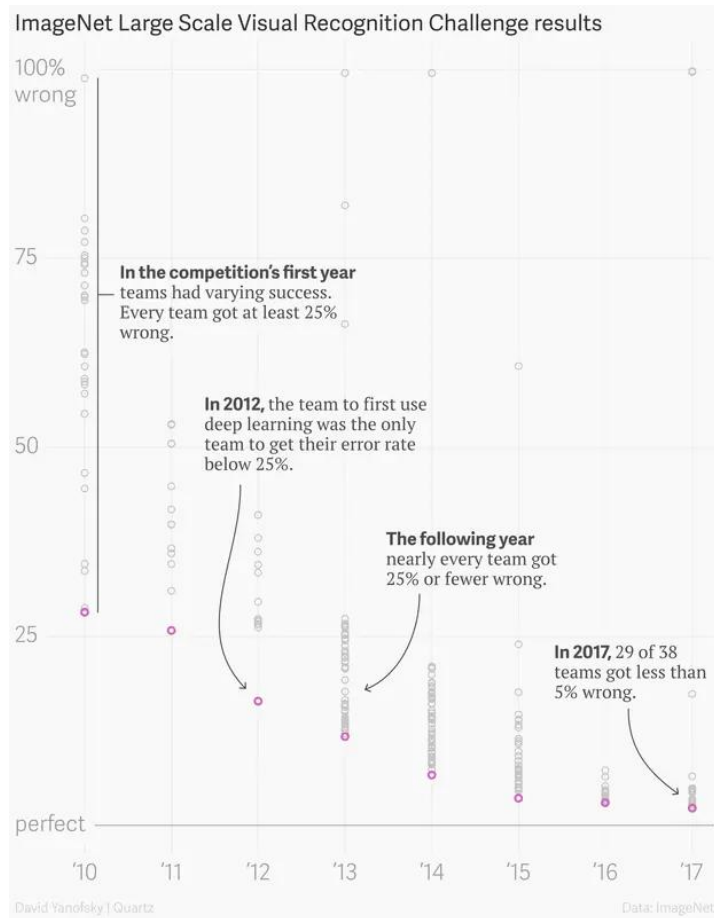
Deep Learning Neural Network



● Hidden Layer

● Output Layer

2012: Deep learning breakthrough

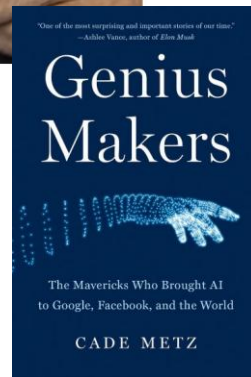


Yoshua Bengio, Geoffrey Hinton, Yann LeCun share 'Nobel of computer science' for work on deep learning

[Emily Chung](#) · CBC News · Posted: Mar 27, 2019 6:00 AM ET | Last Updated: March 27, 2019

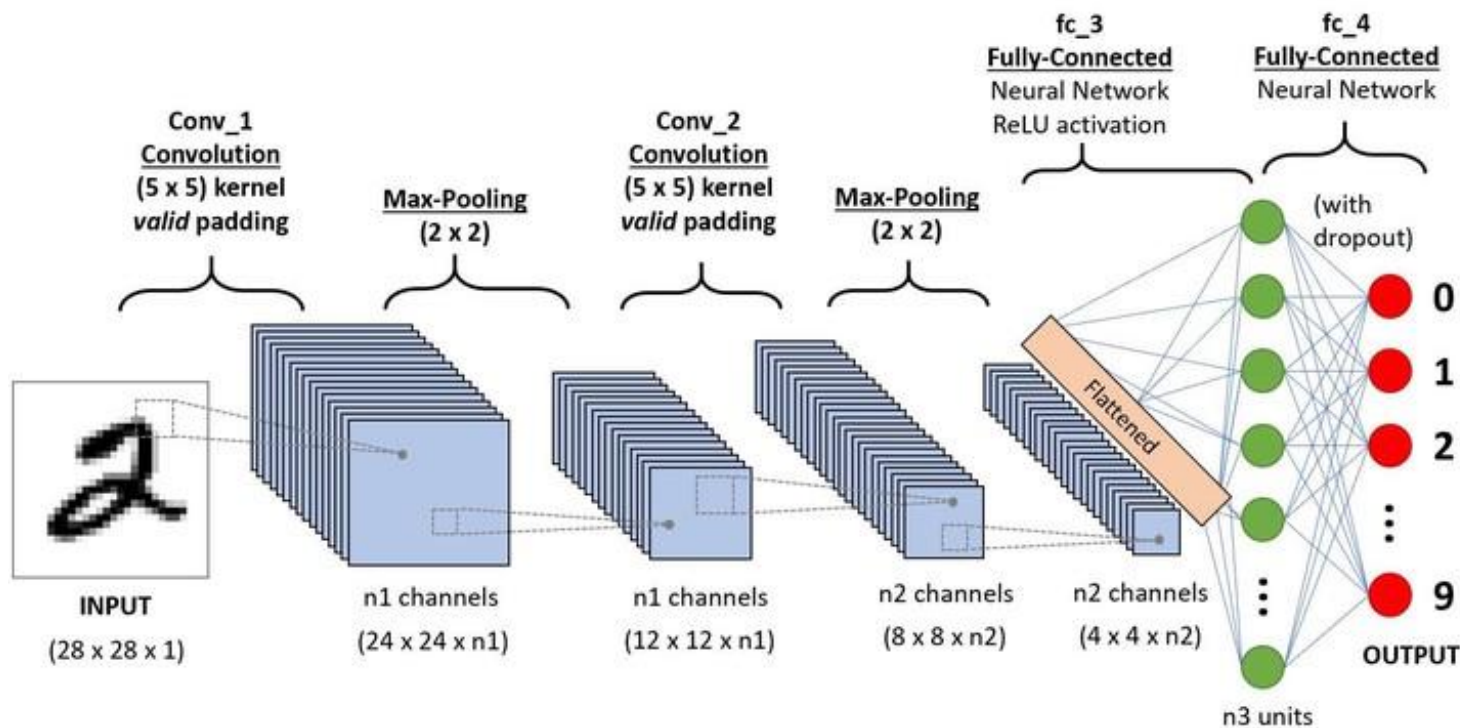


In 2012, Geoffrey Hinton found an algorithm to train multiple layers (deep) with billions of neurons and used it to outperform every other algorithm in image recognition and reached above human recognition levels quickly after that.



2012: Deep Learning for computer vision

- Multi-layer (>3, often 10+) networks can now be trained using convolution.
- Breakthrough in Computer Vision tasks.



Teaching note: Convolution is the key trick: instead of connecting every input to every neuron, a small filter is reused across the whole image — far fewer weights to train.

Padding preserves size of image (each layer conv makes it smaller), Stride deals with convolution on edges, otherwise middle gets more conv. Pooling (max=take most prominent pixels or average = preserves overall picture) reduces spatial size to reduce complexity and computational cost. Softmax at end for multi-class classification.

Convolution

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature



1	1	1	0	0
0	1	1	1	0
0	0	1 _{x1}	1 _{x0}	1 _{x1}
0	0	1 _{x0}	1 _{x1}	0 _{x0}
0	1	1 _{x1}	0 _{x0}	0 _{x1}

Image

4	3	4
2	4	3
2	3	4

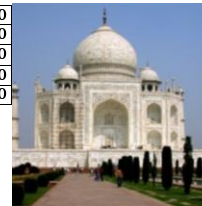
Convolved
Feature

Convolution acts as a filter. Used for feature selection. In this case a AND function with 101,010,101 (in red) = 4

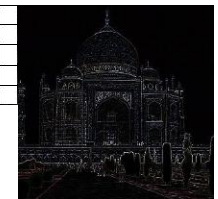
Convolution with 3x3 Filter. Source:

http://deeplearning.stanford.edu/wiki/index.php/Feature_extraction_using_convolution

0	0	0	0	0
0	1	1	1	0
0	1	1	1	0
0	1	1	1	0
0	0	0	0	0

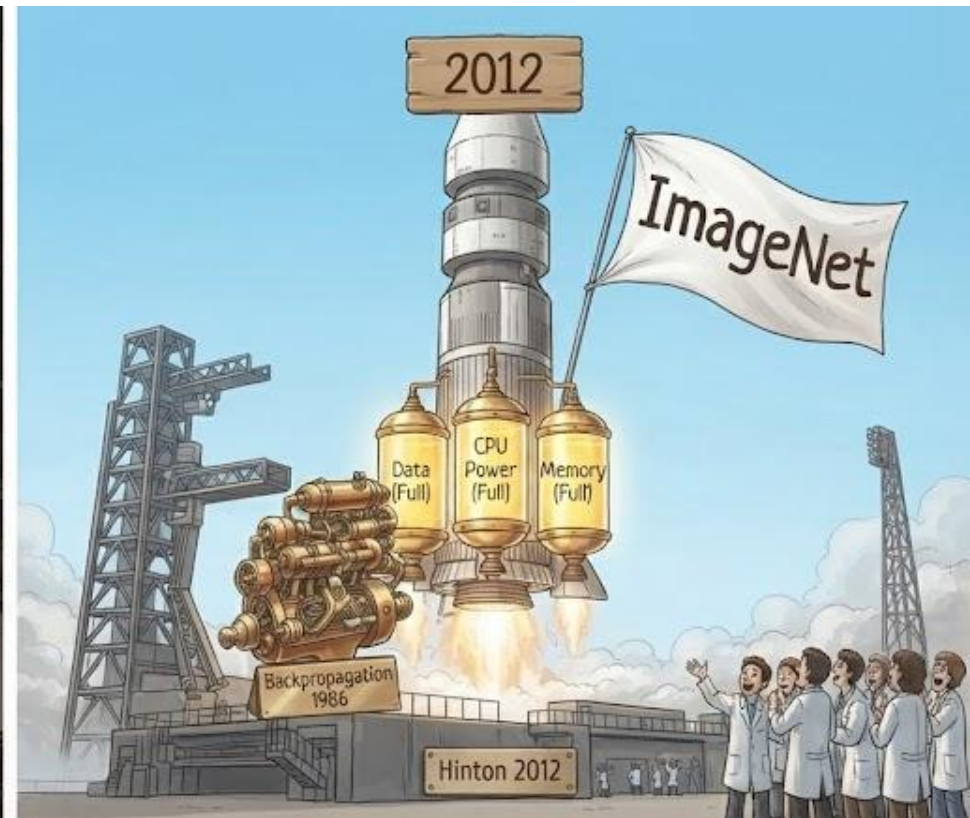


0	1	0		
1	-4	1		
0	1	0		



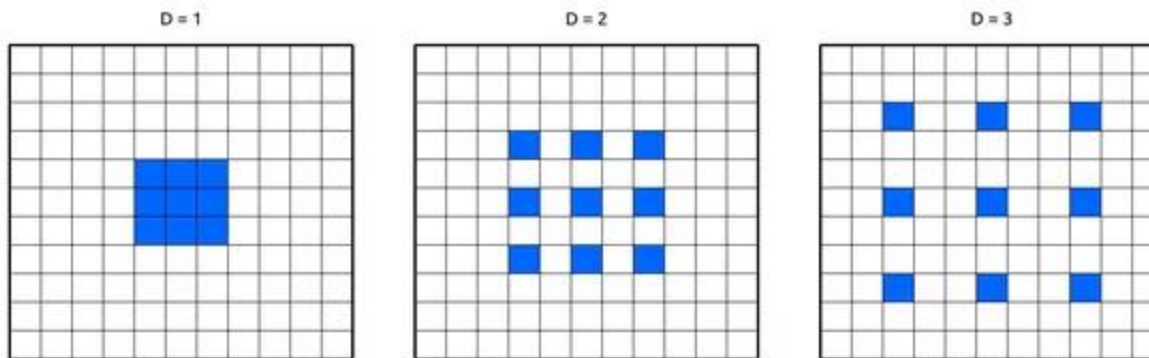
Source: <http://www.wildml.com/2015/11/understanding-convolutional-neural-networks-for-nlp/>

Back propagation Remained



Convolution in Text?

- Context from surroundings, using filters with ratio 1:3 generally work best. Larger ones damage the gradient descent in training.
- Different layers represent different abstraction. Higher layers more semantic representations.
- Use dilated convolutions AKA atrous convolutions AKA convolutions with holes to capture longer term relations without angering the gradient descent.

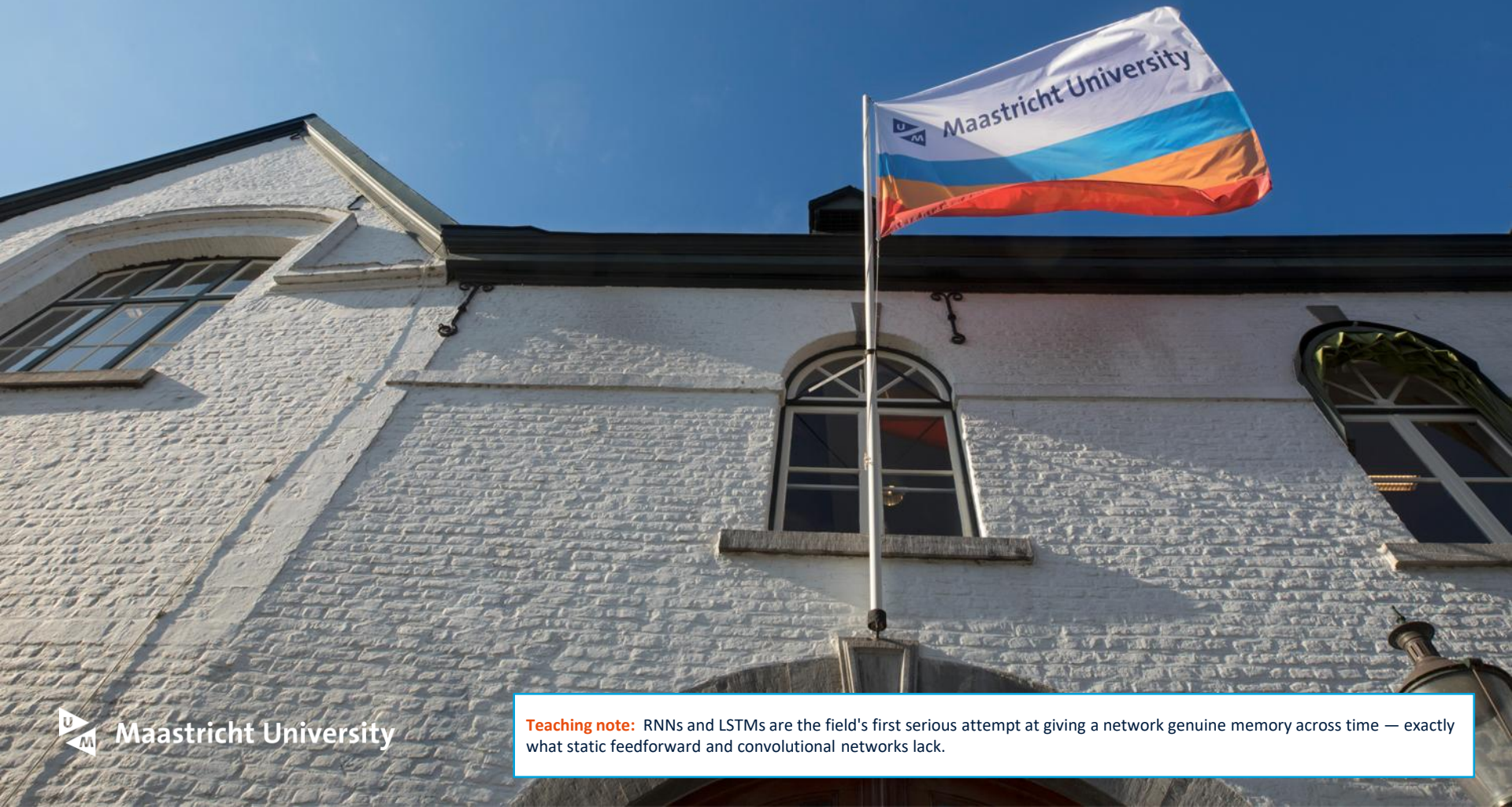


Spatial versus Temporal

- Convolutional Neural Networks work well for spatial problems such as images.
- For temporal (sequential) problems such as language we need a time sensitive aka recurrent neural network architecture. Remember the past (and the future).

Linguistic Context!

Recurrent Neural Networks and LSTM

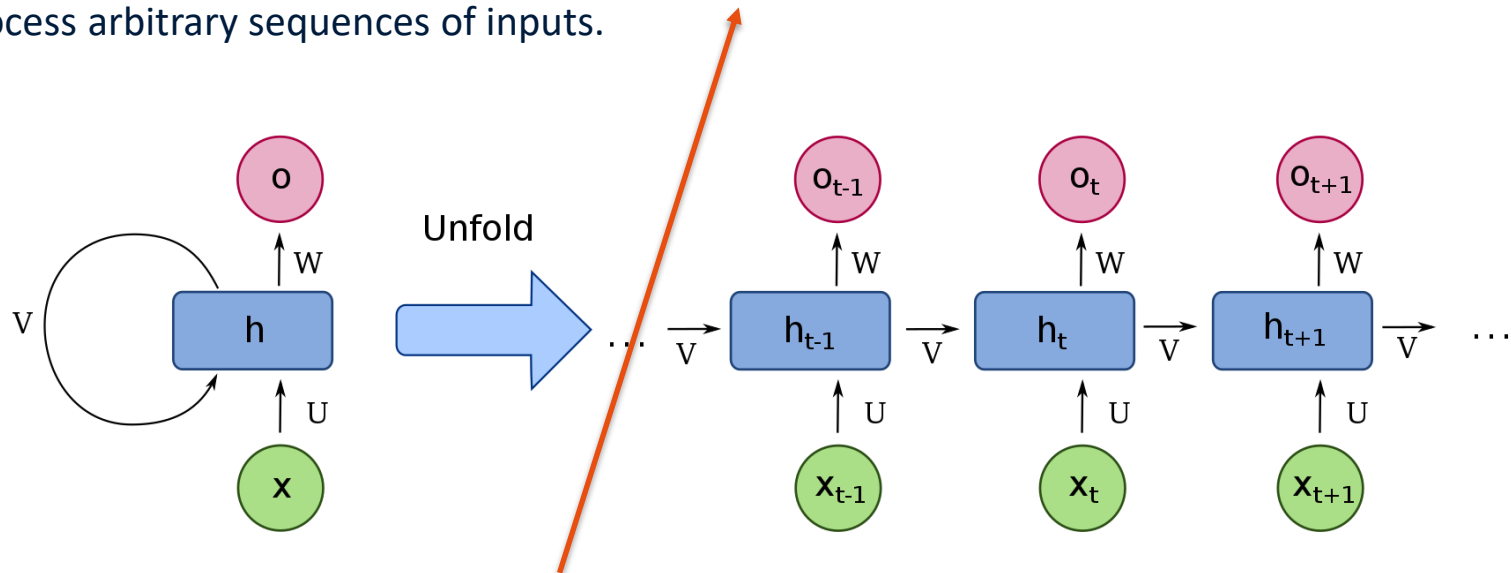


Maastricht University

Teaching note: RNNs and LSTMs are the field's first serious attempt at giving a network genuine memory across time — exactly what static feedforward and convolutional networks lack.

Sequences: Recurrent Neural Networks

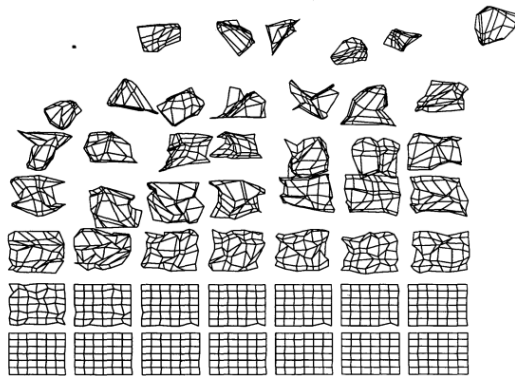
- A recurrent neural network (RNN) is a class of artificial neural networks where connections between nodes can create a cycle, allowing output from some nodes to affect subsequent input to the same nodes.
- This allows it to exhibit temporal dynamic behavior. Derived from feedforward neural networks, RNNs can use their internal state (memory) to process variable length sequences of inputs.
- This makes them applicable to tasks such as unsegmented, connected handwriting recognition, speech recognition or Natural Language Processing.
- Recurrent neural networks are theoretically **Turing complete** and can run arbitrary programs to process arbitrary sequences of inputs.



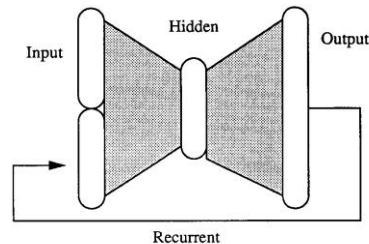
See Chomsky's Hierarchy for more explanation of this notion.

1990-1993: first attempts with back propagation, RNN and self-organizing maps.

Neural Networks in Natural Language Processing and Information Retrieval

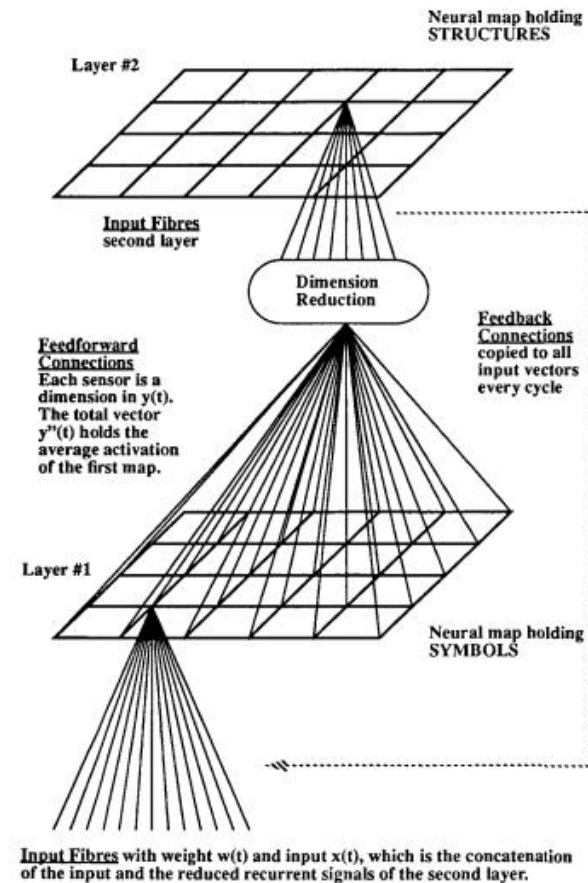


Johannes Cornelis Scholtes

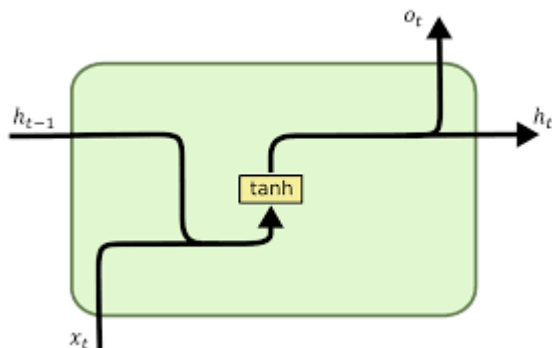
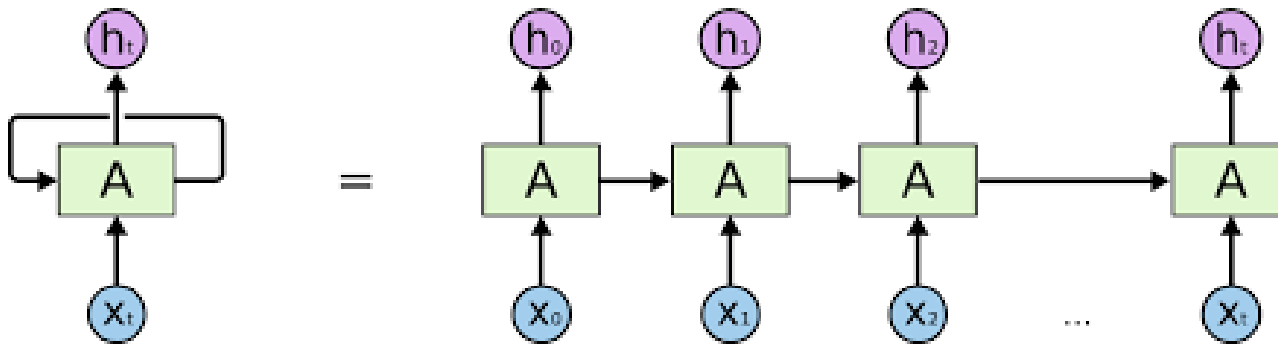


Challenges:

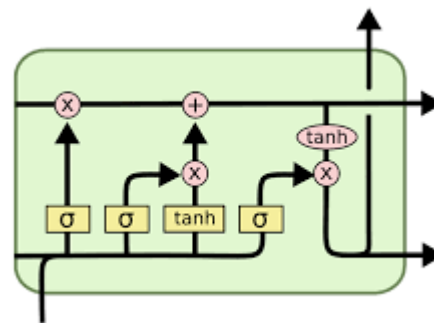
- Not enough data
- Not enough CPU power
- Not enough memory
- Not able to get multi-layer training stable with then known algorithms.



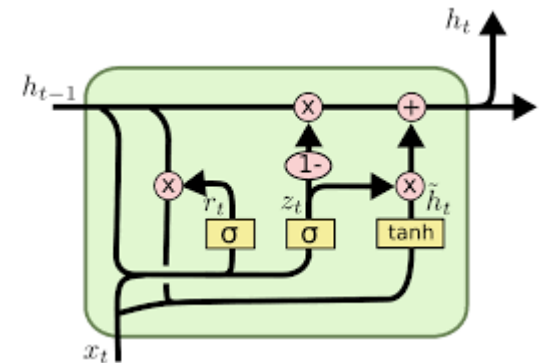
There was a lot of research in RNN, LSTM and GRU



RNN



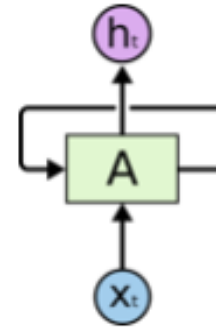
LSTM



GRU

Recurrent Neural Networks

- Our thoughts are persistent.
- We don't start our thinking from scratch every second.
- We understand word based on our understanding of previous words.
- Recurrent Neural Nets address this issue.



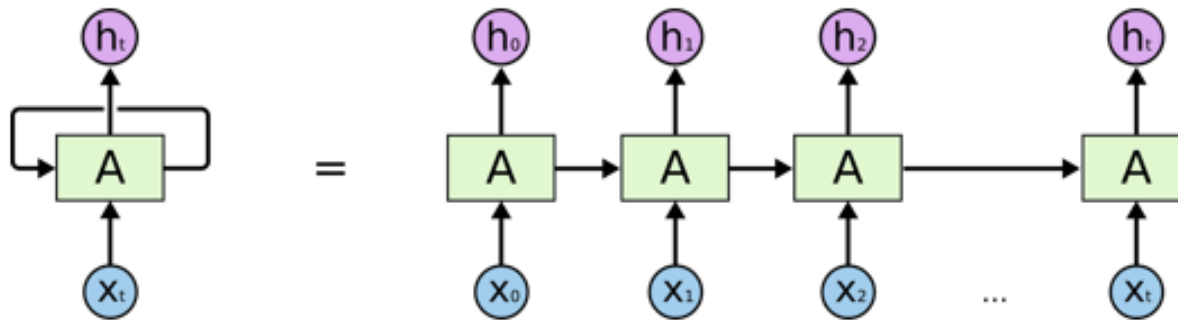
Recurrent Neural Networks have loops.

In the above diagram, a chunk of neural network, **A**, looks at some input x_t and outputs a value h_t

Such a loop allows information to be passed from one step of the network to the next.

From RNN to LSTM

A recurrent neural network can be thought of as multiple copies of the same network, each passing a message to a successor.

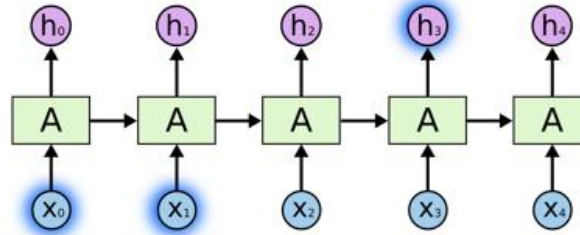


An unrolled recurrent neural network.

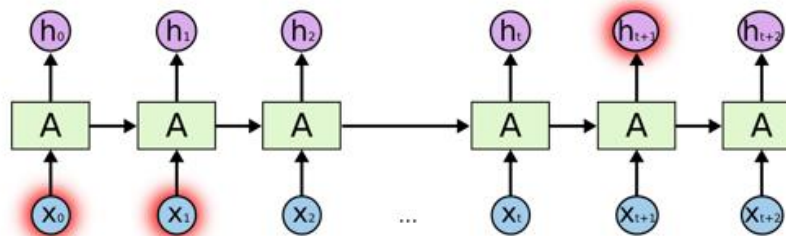
- Ultimately, recurrent neural networks can better handle sequences as they occur in natural language. RNN are the most natural architecture to deal with natural language.
- But how we solve the vanishing/exploding gradient and deal with longer sequences?
- This is where Long Short-Term Memory (LSTM) come in: these models add a memory mechanism to deal with long-term relations.

Why memory?

Long-Term Dependencies: the longer the gap, the harder to predict



Predict “the Clouds are in the *Sky*”

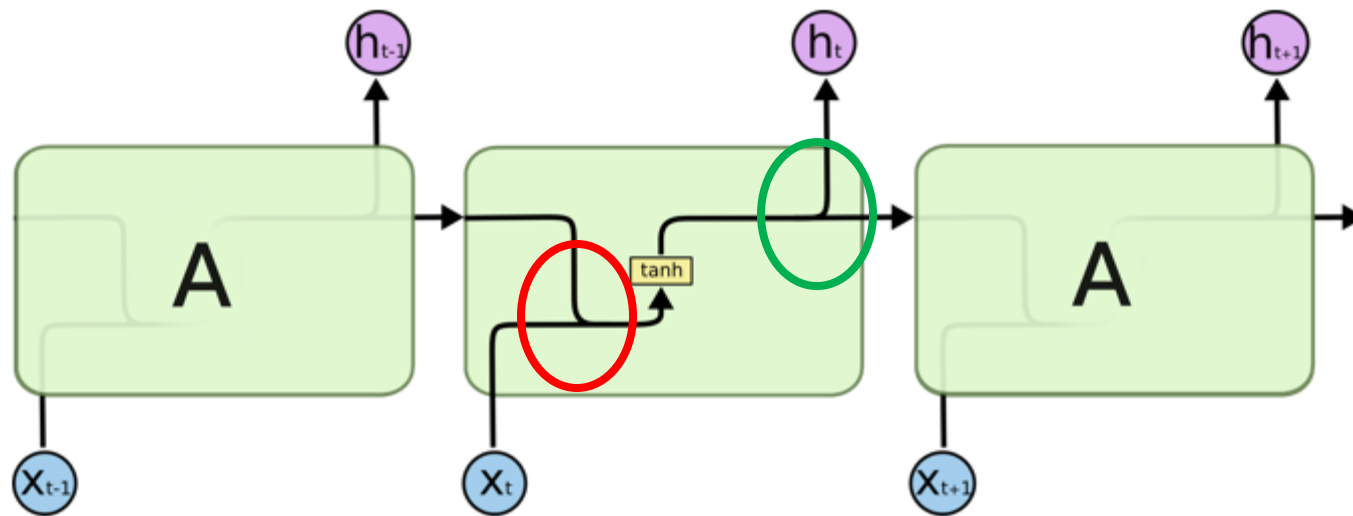


Predict “I grew up in France ... , I speak Fluent *French*”

Teaching note: The France/French example needs memory across many intervening words — exactly what a plain RNN struggles with and LSTM's cell state (next slides) is built to fix.

From RNN to LSTM

All recurrent neural networks have the form of a chain of repeating modules of neural network. In standard RNNs, this repeating module will have a very simple structure, such as a single layer using the Hyperbolic Tangent ($\tanh$).



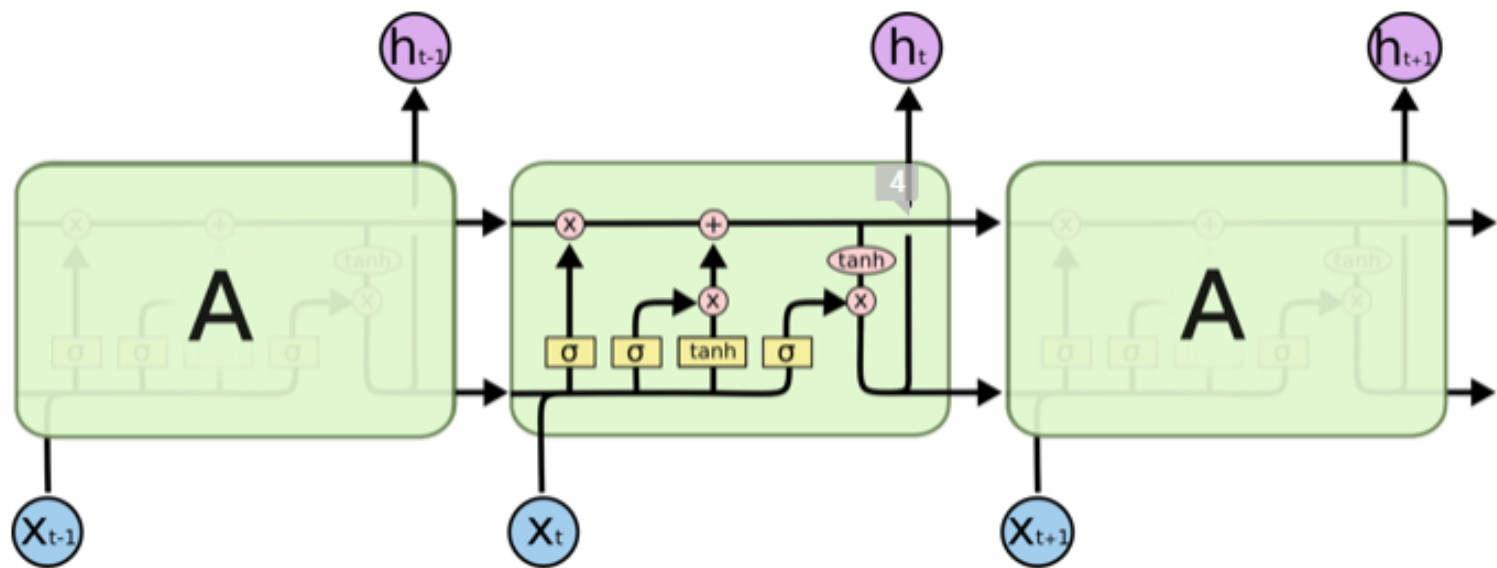
The repeating module in a standard RNN contains a single layer.

 = concatenation vector

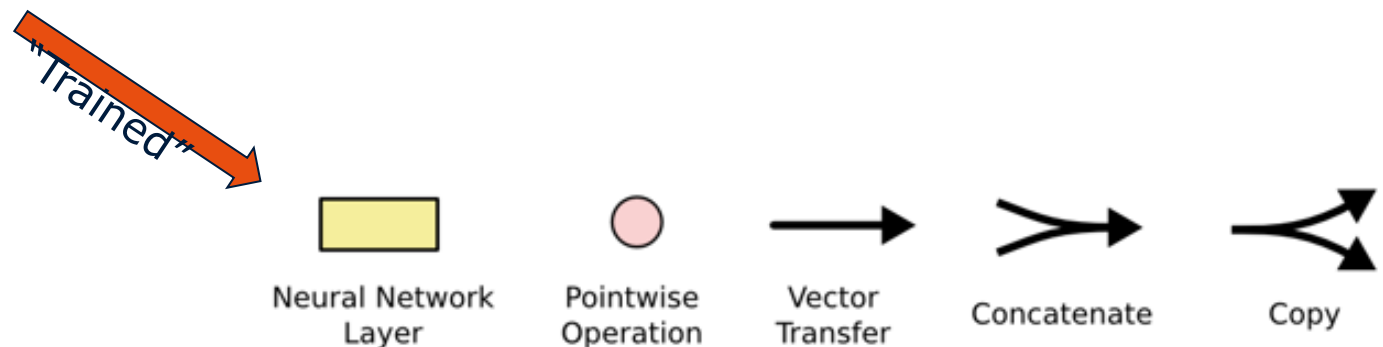
 = content vector is copied

Introduced by Hochreiter & Schmidhuber in 1997

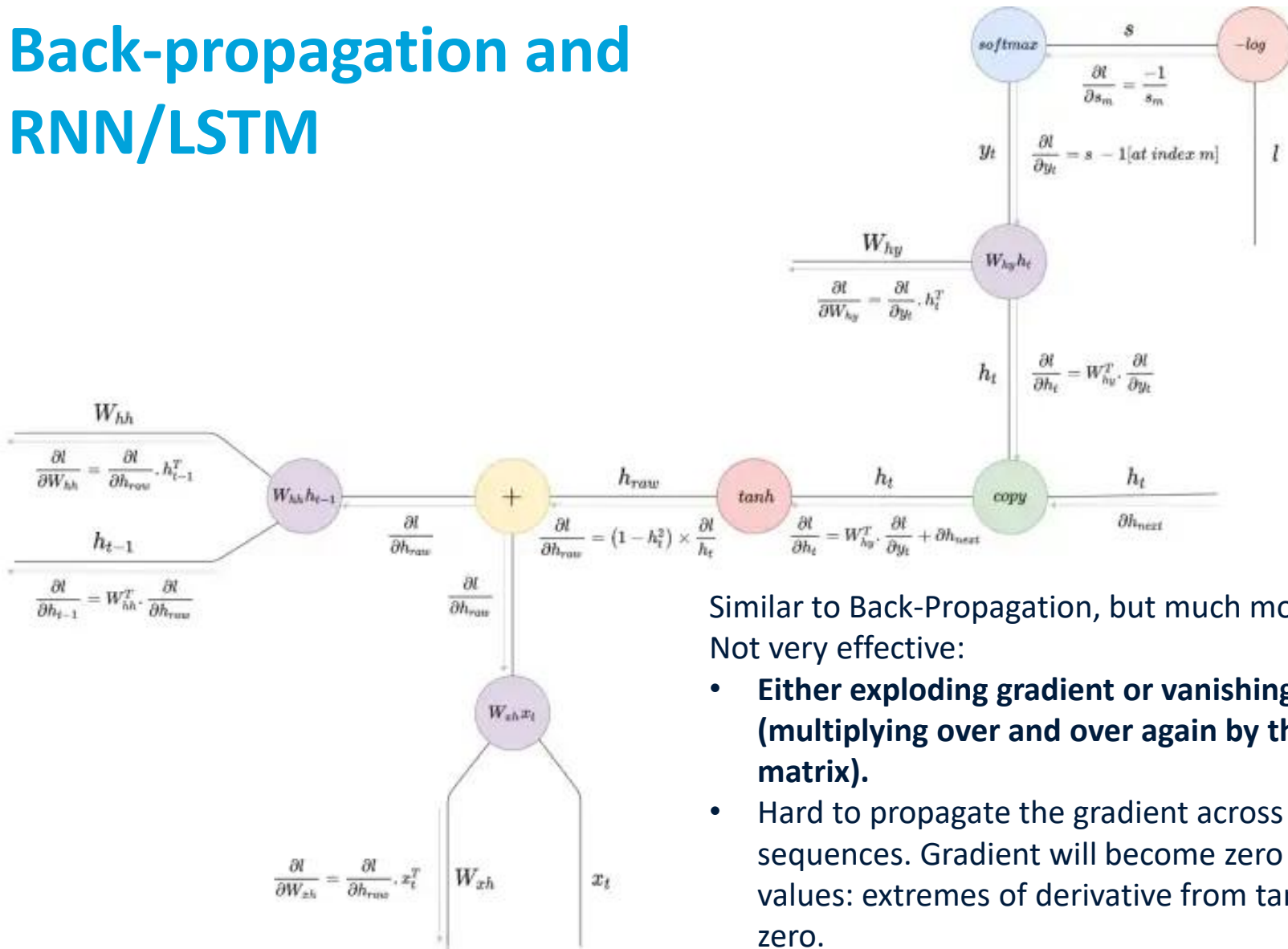
LSTMs also have this chain like structure, but the repeating module has a different structure. Instead of having a single neural network layer, there are four layers (σ , σ , $\tanh$, and σ) interacting in a very special way.



The repeating module in an LSTM contains four interacting layers.



Back-propagation and RNN/LSTM



Similar to Back-Propagation, but much more complex.
Not very effective:

- **Either exploding gradient or vanishing gradient (multiplying over and over again by the weight matrix).**
- Hard to propagate the gradient across long sequences. Gradient will become zero for larger values: extremes of derivative from tangent are zero.
- There are some optimizations (e.g. clipping), but RNN still have problem with long sequences for these reasons.

RNN/LSTM's ...

Teaching note: Keep this list in mind — attention (next lecture) and then the Transformer (Lecture 9) were built specifically to remove the sequential-training bottleneck causing most of these.

- Too many gradients
- Hard to train
- Instable
- Data hungry
- Computationally complex
- 1000s of architectural variations

*Too many problems: there is
room for a better solution!*

LSTMs ...



Questions?



Maastricht University

Teaching note: Recap: McCulloch-Pitts (1943) → Perceptron (1958) → AI winter (1969) → backprop (1986) → deep learning (2012) → RNN/LSTM — each step solved the previous one's core limitation.

Exercise 1

Review the transition from the McCulloch-Pitts neuron (1943) and Rosenblatt's Perceptron (1958) to the formal proof of linear separability limitations published by Minsky and Papert in 1969.

- State the fundamental mathematical differences between a McCulloch-Pitts neuron and a Rosenblatt Perceptron regarding input values, individual input weights, and the threshold activation function.
- The single-layer Perceptron uses the decision rule:
- $y = 1 \Leftrightarrow w_0 + \sum_{i=1}^n w_i x_i \geq 0$
- Write out the system of linear inequalities for the 2-input XOR logic gate ($x_1, x_2 \in \{0,1\}$). Show algebraically why no combination of weights (w_0, w_1, w_2) can satisfy all four conditions simultaneously.
- Why did Minsky and Papert's demonstration that single-layer perceptrons cannot solve non-linear problems trigger the first "AI Winter", and what architectural addition in the 1986 *Parallel Distributed Processing* (PDP) framework finally overcame this limitation?

Exercise 2: Backpropagation, Loss Functions, and Gradient Computations

Modern neural architectures rely on the backpropagation training loop popularized by Rumelhart, Hinton, and Williams (1986).

- Contrast the usage of **Mean Squared Error (MSE)** and **Categorical Cross-Entropy**:
 - Which loss function is suited for continuous regression, and which is designed for discrete classification?
 - Why is the combination of a **Softmax** output layer and Cross-Entropy loss preferred over MSE for multi-class classification?

- Using the chain rule derivation from the lecture:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_j} \cdot \frac{\partial o_j}{\partial \text{net}_j} \cdot \frac{\partial \text{net}_j}{\partial w_{ij}} = o_i \delta_j$$

Given a single output neuron j using a standard sigmoid activation $\varphi(z) = \frac{1}{1+e^{-z}}$ (where $\varphi'(z) = o_j(1 - o_j)$) and squared error loss $E = \frac{1}{2}(o_j - t_j)^2$, derive the exact expression for δ_j .

- Explain how error blame δ_j is backpropagated to a hidden neuron from the subsequent layer k . Why do saturating activations like sigmoid or tanh cause the vanishing gradient problem in deep multi-layer networks, and how do modern activations like ReLU or GELU help mitigate this?

Exercise 3: Spatial vs. Temporal Modeling (CNNs to RNNs & LSTMs)

Trace the shift from spatial deep learning (AlexNet, 2012) to sequential deep learning models designed for natural language.

- Convolutional Neural Networks (CNNs) achieved superhuman performance on ImageNet using spatial operations (convolution, stride, padding, pooling). Explain why applying 1D CNNs or dilated convolutions to text struggled to serve as a complete solution for modeling natural language syntax and semantics.
- Standard Recurrent Neural Networks (RNNs) pass a hidden state vector h_t through time:
 - How does an RNN theoretically achieve Turing completeness on arbitrary input sequences?
 - When training an RNN using Backpropagation Through Time (BPTT), why does repeatedly multiplying hidden state weight matrices across long sentences cause vanishing or exploding gradients?
- Hochreiter and Schmidhuber (1997) introduced the Long Short-Term Memory (LSTM) network to solve this degradation. How does the internal gating mechanism ($\sigma, \sigma, \tanh, \sigma$) and cell state highway (C_t) allow an LSTM to retain long-term dependencies (e.g., matching "*I grew up in France*" with "*I speak fluent French*") far better than a vanilla RNN? Why did LSTMs still remain bottlenecked by computational complexity and sequential training constraints?

Discussion Lecture: History of Deep Learning



On the Illusion of Biology: Artificial Neurons vs. Real Brains

In 1958, the New York Times reported that the Perceptron was an embryo of an electronic computer expected to 'think', 'grow wiser', and even become 'conscious'. Yet beneath that rhetoric, Rosenblatt's device was simply a linear classifier summing weighted inputs with a step threshold.

Today, we build deep neural networks with billions of parameters, but the fundamental forward-pass unit remains a weighted sum followed by an activation function.

Are modern deep learning systems actually modeling the biological cognition of the brain as early cyberneticists envisioned, or did we just repurpose neurobiological terminology to make multi-variable calculus and linear algebra sound intelligent? If an artificial neuron shares almost nothing with a biological cell except its name, why did the brain metaphor remain so resilient across eighty years of AI development?



On the Linear Separability Trauma: The Cost of Discarding Promising Paradigms

In 1969 Minsky and Papert proved that a single-layer perceptron cannot compute XOR. Funding dried up and neural network research entered a winter that lasted more than a decade, although the way out (hidden layers trained by the chain rule) was mathematically within reach the whole time.

Did the field abandon connectionism because the limitation was insurmountable, or because a rival school convinced the community that it was? What decides which critiques kill a paradigm and which are absorbed by it?

Today deep learning has its own well-documented failures: hallucination, lack of grounding, weak reasoning. Are we at risk of prematurely declaring another paradigm dead, or of refusing to declare this one dead because too much depends on it?

On Depth: When Does More Become Different?

The breakthrough of 2012 added no new operation. It stacked the same one, a weighted sum and a nonlinearity, many more times than before, and called the result “deep”.

When does more of the same become something different? A single perceptron draws a line; two layers draw any shape; a hundred layers recognise a face. Is that a change in kind, or only in quantity, and does the distinction matter?

If understanding can emerge from repetition of a trivial step, what, if anything, is left of the idea that understanding requires a special ingredient? And if nothing emerges, what are we looking at when a deep network succeeds?